

SmartClaim AI: Pipeline Code Explanation

A Detailed Guide for University Presentation

1. Overview of the Pipeline

The notebook **SmartClaim_Colab_Pipeline.ipynb** represents an end-to-end Machine Learning pipeline. It processes Arabic traffic accident descriptions and predicts realistic vehicle repair costs. The pipeline follows a 'Hybrid Expert-Steered ML' approach, meaning it combines statistical Machine Learning (like XGBoost or Random Forest) with human-encoded rules (Expert System) to prevent unrealistic cost predictions.

2. Arabic Feature Extraction Engine

The code defines a class called `ArabicAccidentFeatureExtractor`. Its purpose is to parse free-text Arabic accident descriptions and extract structured, numerical features.

How it works:

- It defines dictionaries of Arabic keywords categorized by Severity (e.g., Very Light, Minor, Critical), Impact Direction (Front, Rear, Side), and specific car parts (e.g., Door, Chassis, Bumper).
- The `extract_features()` method scans the text and flags the presence of these keywords as 1 or 0 (binary flags), and counts the number of distinct parts damaged.
- The `transform()` method converts these flags into a dense NumPy array that Machine Learning models can understand.

3. Semantic Text Embedding

Machine learning models cannot read text directly; they require numbers. The `TextEmbedder` class uses the HuggingFace `sentence-transformers` library, specifically the **paraphrase-multilingual-mpnet-base-v2** model.

- **Dense Embeddings:** It transforms each Arabic sentence into a 768-dimensional dense vector representing its semantic meaning.
- **Feature Concatenation:** It concatenates this 768-dimensional vector with the 6 structured features from our Arabic Feature Extractor, resulting in a rich 774-dimensional hybrid feature vector.
- **Caching:** To save time in Colab, it caches embeddings to disk using `joblib`.

4. Data Preparation

The `load_data()` function dynamically searches for the dataset `smartclaim_expanded_dataset.xlsx`. It uses the Pandas library to read the Excel file, renames columns to 'text' and 'cost', drops missing

values, and strips whitespace. Then, `split_data()` divides the data into Training (80%) and Testing (20%) sets to ensure the model can be evaluated on unseen data.

5. Model Training and Selection

The `train_models()` function implements a competitive training phase:

- It takes the training set and splits it further (90% training, 10% internal validation).
- It trains two robust models: **Random Forest Regressor** (an ensemble of decision trees) and **XGBoost Regressor** (a gradient boosting algorithm).
- It compares both models using Mean Absolute Error (MAE) on the validation set. The model with the lower error is selected as the 'Winner' and saved to disk using `joblib.dump()`.

6. Model Evaluation

The `evaluate_model()` function tests the winning model on the 20% hold-out test set. It calculates metrics like R-Squared (R²), MAE, and RMSE. Crucially, it uses `matplotlib` and `seaborn` to generate diagnostic plots:

- **Actual vs Predicted Plot:** Shows how closely predictions follow the ideal diagonal line.
- **Residuals Distribution:** Checks if errors are normally distributed.
- **Feature Importance:** Displays which variables had the most impact on the prediction.

7. The 7-Tier Expert Cost Estimation Engine

This is the most innovative part of the code. The `predict_cost()` function prevents the ML model from producing generic averages (a flaw known as 'bucketing bias').

The Logic:

1. **Tier Assignment:** Based on the rule-based extracted keywords, the accident is strictly assigned to one of 7 cost tiers (e.g., 'Very Light' -> 0-1,000 SAR, 'Critical' -> 25,000-60,000 SAR).
2. **Relative Positioning:** The raw ML prediction is normalized into a ratio (0.1 to 0.9). This ratio decides where the final cost sits *within* the assigned tier bounds.
3. **Penalties and Boosts:** If scratches are detected, the cost shifts lower in the tier. If many parts are damaged, the cost shifts higher.
4. **Smart Overrides:** If the text explicitly says 'no damage', a hard cap is enforced.

8. Full Pipeline Execution Flow

The `run_pipeline()` function ties everything together in three phases:

- **Phase 1:** Train candidates and select the winner.

- **Phase 2:** Validate the winner on training data to verify learning.
- **Phase 3:** Evaluate strictly on the hold-out test data to prove real-world generalization capability.

Generated by SmartClaim AI Assistant